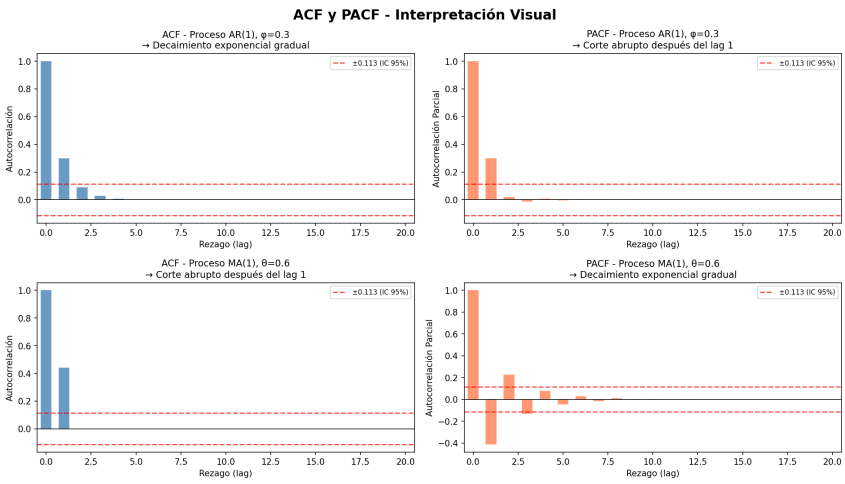
Modelos SARIMA

Guía Completa y Accesible

con Aplicaciones en Series de Tiempo Financieras

Temas cubiertos en esta guía:

- Forma general del modelo SARIMA
- ACF, PACF y cómo interpretarlos
- Criterios de información: AIC y BIC
- Identificación del modelo paso a paso
- 3 ejemplos completos en Python
- Datos: Acción, Tipo de cambio, Petróleo



Abril 2026

0. Contents

1 ¿Qué es un modelo SARIMA? 2
1.1 La Notación General 2
1.2 La Ecuación Matemática Completa 2

2 ACF y PACF: Las Herramientas de Diagnóstico 3
2.1 ¿Cómo interpretar las gráficas? 4
2.2 Tabla de Identificación del Modelo 5
2.3 Las Bandas de Confianza 5

3 Criterios de Selección: AIC y BIC 5

4 Proceso de Modelado Paso a Paso 6

5 Ejemplo 1: Precio de Acción (tipo AAPL) — ARIMA(1,1,1) 7
5.1 Código Python — Ejemplo 1 7

6 Ejemplo 2: Tipo de Cambio USD/MXN — SARIMA(1,1,1)(1,1,1)$_{12}$ 10
6.1 Código Python — Ejemplo 2 10

7 Ejemplo 3: Precio del Petróleo WTI — ARIMA(2,1,2) con Pronóstico 13
7.1 Código Python — Ejemplo 3 13

8 Resumen de Criterios de Diagnóstico 16

1. ¿Qué es un modelo SARIMA?

Definición General

SARIMA significa *Seasonal AutoRegressive Integrated Moving Average*. Es la versión **estacional** del modelo ARIMA, que a su vez generaliza a AR y MA. Se usa para modelar y pronosticar series de tiempo que pueden tener:

- **Tendencia** (necesita diferenciación)
- **Estacionalidad** (patrones que se repiten cada S períodos)
- **Autocorrelación** (el pasado explica el presente)

1.1. La Notación General

La notación completa es:

$$\boxed{(S)\text{ARIMA}(p,\, d,\, q)\,(P,\, D,\, Q)_S}$$

Donde cada letra tiene un significado preciso:

Parámetro	Parte NO estacional	Qué controla
p	Orden autorregresivo	Cuántos **rezagos pasados** de la serie usamos
d	Grado de integración	Cuántas veces **diferenciamos** para hacer la serie estacionaria
q	Orden de media móvil	Cuántos **errores pasados** incluimos
P	AR estacional	Rezagos estacionales en la parte AR
D	Diferenciación estacional	Diferenciación para eliminar la estacionalidad
Q	MA estacional	Errores estacionales en la parte MA
S	Período estacional	Cada cuántos períodos se repite el patrón

La letra "S" al inicio

La "(S)" al inicio de SARIMA simplemente indica que el modelo **incluye componentes estacionales**. Si no hay estacionalidad, es simplemente ARIMA(p, d, q) y los parámetros $(P, D, Q)_S$ son todos cero.

1.2. La Ecuación Matemática Completa

La ecuación del modelo SARIMA$(p, d, q)(P, D, Q)_S$ se escribe usando operadores de rezago B, donde $B^k y_t = y_{t-k}$:

$$\Phi_P(B^S)\,\phi_p(B)\,(1-B)^d\,(1-B^S)^D\,y_t = \Theta_Q(B^S)\,\theta_q(B)\,\varepsilon_t$$

Donde:

- $\phi_p(B) = 1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p$ (polinomio AR no estacional)
- $\theta_q(B) = 1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q$ (polinomio MA no estacional)
- $\Phi_P(B^S) = 1 - \Phi_1 B^S - \cdots - \Phi_P B^{PS}$ (polinomio AR estacional)
- $\Theta_Q(B^S) = 1 + \Theta_1 B^S + \cdots + \Theta_Q B^{QS}$ (polinomio MA estacional)
- $\varepsilon_t \sim \mathrm{IID}(0, \sigma^2)$ (ruido blanco)

Ejemplo simplificado: ARIMA(1,1,0)

Si tenemos $p = 1,\ d = 1,\ q = 0$ sin estacionalidad, la ecuación es:

$$(1 - \phi_1 B)(1 - B)\,y_t = \varepsilon_t$$

Que al expandir se convierte en:

$$y_t - y_{t-1} = \phi_1(y_{t-1} - y_{t-2}) + \varepsilon_t$$

Es decir: el **cambio de hoy** depende del **cambio de ayer** más un error aleatorio.

2. ACF y PACF: Las Herramientas de Diagnóstico

Definiciones Clave

- **ACF** (*Autocorrelation Function*): Mide la correlación entre y_t y y_{t-k} para cada rezago k. Incluye efectos directos e **indirectos**.
- **PACF** (*Partial Autocorrelation Function*): Mide la correlación entre y_t y y_{t-k} eliminando el efecto de los rezagos intermedios. Solo mide el efecto **directo**.

2.1. ¿Cómo interpretar las gráficas?

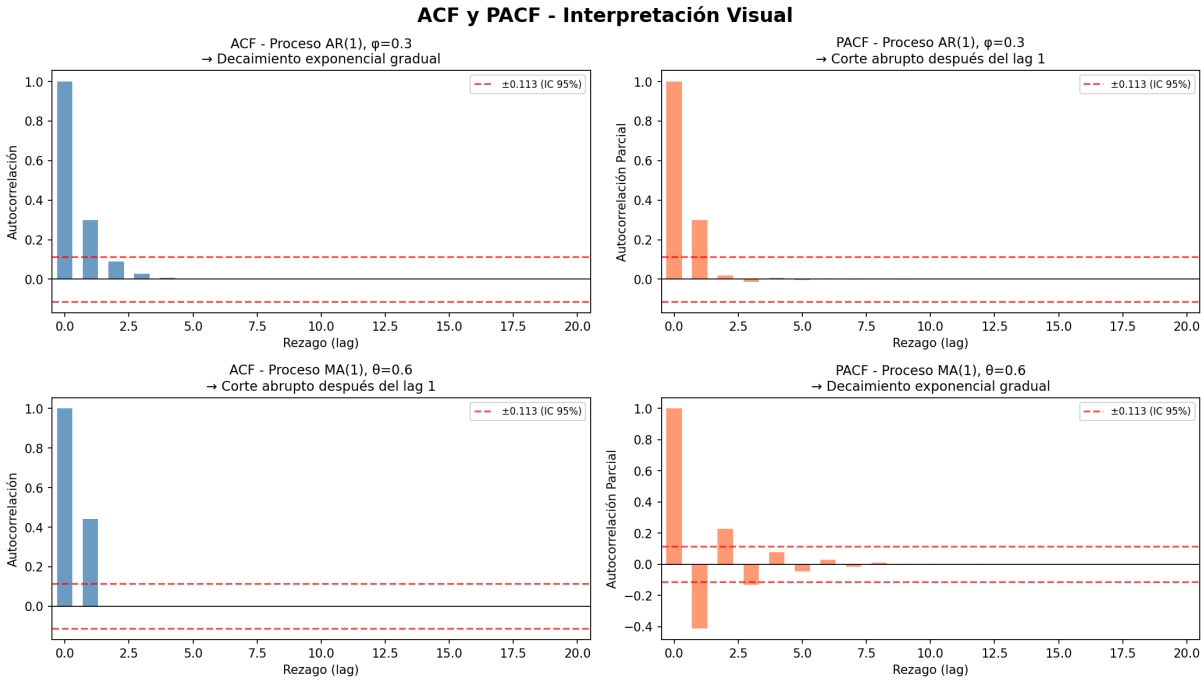


Figure 1: Comparación de ACF y PACF para procesos AR(1) y MA(1). Las líneas rojas punteadas representan bandas de confianza al 95%.

Interpretación de la ACF

1. **Las barras dentro de las líneas rojas** son estadísticamente NO significativas (ruido)
2. **Las barras fuera de las líneas rojas** indican correlación significativa a ese rezago
3. **Decaimiento gradual** (exponencial o en ola): sugiere un proceso **AR**
4. **Corte abrupto** después del rezago q: sugiere un proceso **MA(q)**
5. **Picos en rezagos 12, 24, 36...**: hay **estacionalidad** con $S = 12$ (datos mensuales)
6. **Muchas barras significativas que no decaen**: la serie NO es estacionaria, necesita diferenciación

Interpretación de la PACF

1. **Corte abrupto** después del rezago p: sugiere un proceso **AR(p)**
2. **Decaimiento gradual**: sugiere un proceso **MA**
3. El valor en el **rezago 1** siempre coincide con la ACF en rezago 1
4. Un **pico en rezago** S y corte después: sugiere AR estacional de orden 1

2.2. Tabla de Identificación del Modelo

Tabla de Identificación: ACF y PACF → Modelo ARIMA/SARIMA

Proceso	ACF	PACF	Modelo a usar
AR(p)	Decae gradualmente (exponencial u oscilante)	Se corta abruptamente después del lag p	ARIMA(p, d, 0)
MA(q)	Se corta abruptamente después del lag q	Decae gradualmente (exponencial u oscilante)	ARIMA(0, d, q)
ARMA(p,q)	Decae gradualmente después del lag q	Decae gradualmente después del lag p	ARIMA(p, d, q)
Estacional AR(P)	Decae en lags kS (k=1,2,3...)	Se corta en lag S	SARIMA(·)(P,D,0)_S
Estacional MA(Q)	Se corta en lag S	Decae en lags kS	SARIMA(·)(0,D,Q)_S

Figure 2: Guía rápida para identificar el tipo de modelo basándose en los patrones de ACF y PACF.

2.3. Las Bandas de Confianza

Las líneas rojas punteadas en la ACF/PACF son **bandas de confianza al 95%** y se calculan como:

$$\pm\frac{1.96}{\sqrt{T}}$$

Donde T es el número de observaciones. Si un valor cae **fuera** de estas bandas, hay evidencia estadística de que esa autocorrelación es significativa.

3. Criterios de Selección: AIC y BIC

Una vez identificados varios modelos candidatos, usamos criterios estadísticos para elegir el mejor:

Criterio	Fórmula	Penalización
AIC	$-2\ln(\hat{L}) + 2k$	Suave — prefiere modelos con más parámetros
BIC	$-2\ln(\hat{L}) + k\ln(T)$	Fuerte — prefiere modelos más simples

Donde k = número de parámetros, T = número de observaciones, $\hat{L}$ = verosimilitud maximizada.

Regla de Oro

Siempre elige el modelo con el AIC o BIC más bajo. Si AIC y BIC no coinciden, el BIC favorece modelos más parsimoniosos (simples). En finanzas, generalmente se prefiere el BIC.

4. Proceso de Modelado Paso a Paso

Los 6 Pasos del Modelado SARIMA

1. **Visualizar** la serie: ¿hay tendencia? ¿estacionalidad? ¿varianza cambiante?
2. **Transformar** si es necesario: logaritmo para estabilizar varianza
3. **Diferenciar**: probar estacionariedad con prueba Dickey-Fuller. Determinar d y D
4. **Graficar ACF y PACF** de la serie estacionaria: identificar p, q, P, Q
5. **Estimar** modelos candidatos y comparar con AIC/BIC
6. **Diagnóstico** de residuales: deben ser ruido blanco (sin autocorrelación)

5. Ejemplo 1: Precio de Acción (tipo AAPL) — ARIMA(1,1,1)

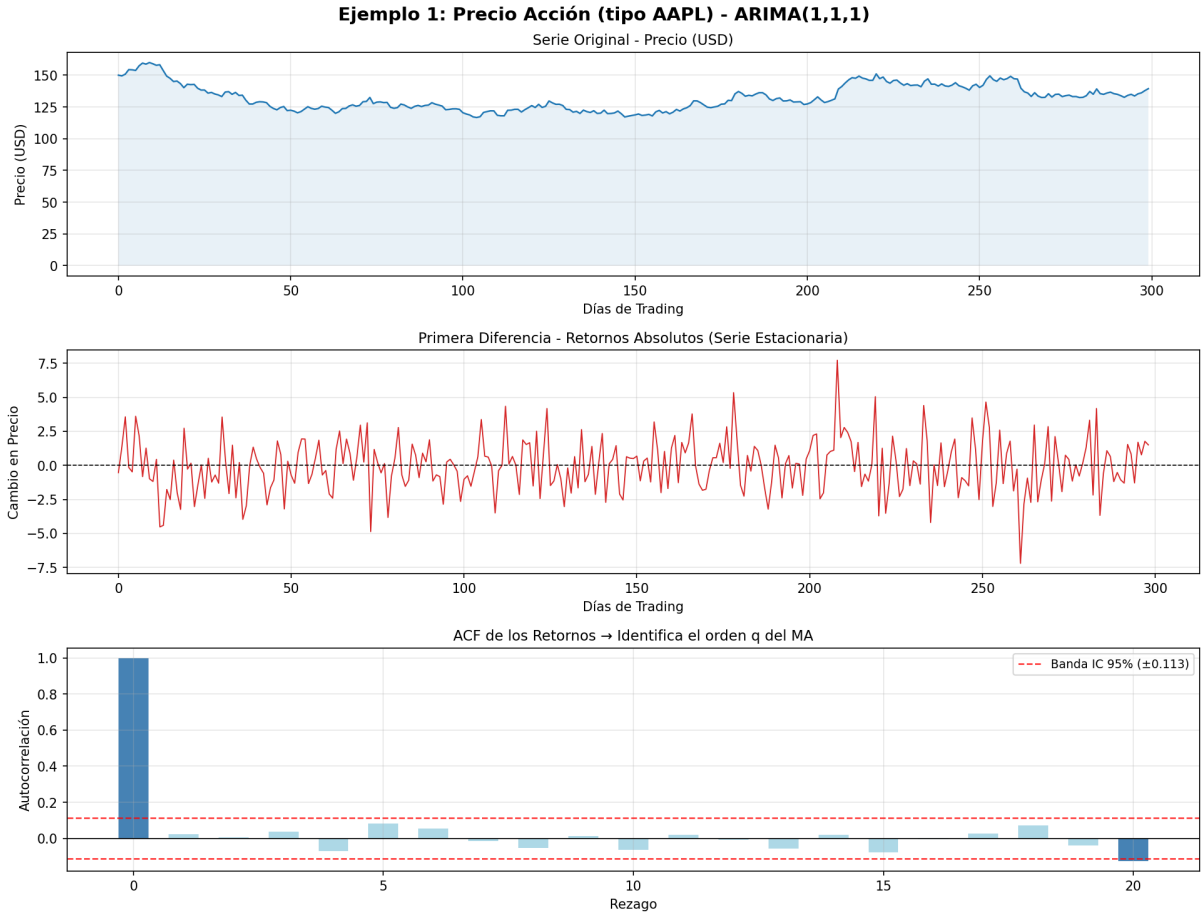


Figure 3: Serie de precio accionario, primera diferencia (retornos) y ACF de los retornos.

¿Por qué ARIMA(1,1,1) para precios de acciones?

- $d = 1$: Los precios no son estacionarios (tienen tendencia), pero los **retornos** (cambios) sí lo son. Una diferenciación los hace estacionarios.
- $p = 1$: La PACF muestra un pico significativo en el rezago 1 y luego cae. El rendimiento de ayer tiene efecto sobre el de hoy.
- $q = 1$: La ACF muestra un pico en rezago 1 y luego oscila, indicando un componente MA.
- **Sin estacionalidad**: Datos diarios de acciones no suelen mostrar estacionalidad fuerte.

5.1. Código Python — Ejemplo 1

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
```

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller

# ================================================================
# PASO 1: Descargar datos (en produccion usar yfinance)
# pip install yfinance
# import yfinance as yf
# data = yf.download("AAPL", start="2020-01-01", end="2024-12-31")
# precio = data["Close"]
#
# Para este ejemplo, simulamos datos similares:
# ================================================================
np.random.seed(42)
n = 300
phi, theta, sigma = 0.3, -0.2, 0.015
eps = np.random.normal(0, sigma, n)
returns = np.zeros(n)
for t in range(1, n):
    returns[t] = phi*returns[t-1] + eps[t] + theta*eps[t-1]
precio = pd.Series(150 * np.cumprod(1 + returns),
                   name="AAPL_Simulado")

# PASO 2: Prueba de estacionariedad (Dickey-Fuller)
resultado_adf = adfuller(precio)
print(f"ADF Statistic: {resultado_adf[0]:.4f}")
print(f"p-value: {resultado_adf[1]:.4f}")
# Si p-value > 0.05, la serie NO es estacionaria --> diferenciar

# PASO 3: Primera diferencia
retornos = precio.diff().dropna()
resultado_adf2 = adfuller(retornos)
print(f"ADF retornos: p-value = {resultado_adf2[1]:.4f}")
# Ahora p-value < 0.05 --> serie estacionaria con d=1

# PASO 4: Graficar ACF y PACF
fig, axes = plt.subplots(2, 1, figsize=(12, 6))
plot_acf(retornos, lags=20, ax=axes[0])
axes[0].set_title("ACF de los Retornos")
plot_pacf(retornos, lags=20, ax=axes[1])
axes[1].set_title("PACF de los Retornos")
plt.tight_layout()
plt.show()
# Interpretacion: PACF corta en lag 1 --> AR(1)
#                 ACF decae con pico en lag 1 --> MA(1)
# Modelo candidato: ARIMA(1,1,1)

# PASO 5: Estimar el modelo
modelo = ARIMA(precio, order=(1, 1, 1))
resultado = modelo.fit()
print(resultado.summary())
print(f"\nAIC: {resultado.aic:.2f}")
print(f"BIC: {resultado.bic:.2f}")

# PASO 6: Pronostico a 20 dias
pronostico = resultado.get_forecast(steps=20)
media_pred = pronostico.predicted_mean
ic_pred = pronostico.conf_int()

```

```python
plt.figure(figsize=(12, 5))
plt.plot(precio[-60:].values, label="Precio historico", color="steelblue")
plt.plot(range(60, 80), media_pred.values,
         label="Pronostico", color="red", linestyle="--")
plt.fill_between(range(60, 80),
                 ic_pred.iloc[:, 0], ic_pred.iloc[:, 1],
                 alpha=0.2, color="red", label="IC 95%")
plt.legend()
plt.title("Pronostico ARIMA(1,1,1) - Precio AAPL")
plt.ylabel("Precio (USD)")
plt.show()

# PASO 7: Diagnostico de residuales
residuales = resultado.resid
resultado.plot_diagnostics(figsize=(12, 8))
plt.tight_layout()
plt.show()
# Los residuales deben: ser ruido blanco, distribucion normal,
# ACF sin autocorrelacion significativa
```

Listing 1: Ejemplo 1: ARIMA(1,1,1) para precio de acción

6. Ejemplo 2: Tipo de Cambio USD/MXN — SARIMA(1,1,1)(1

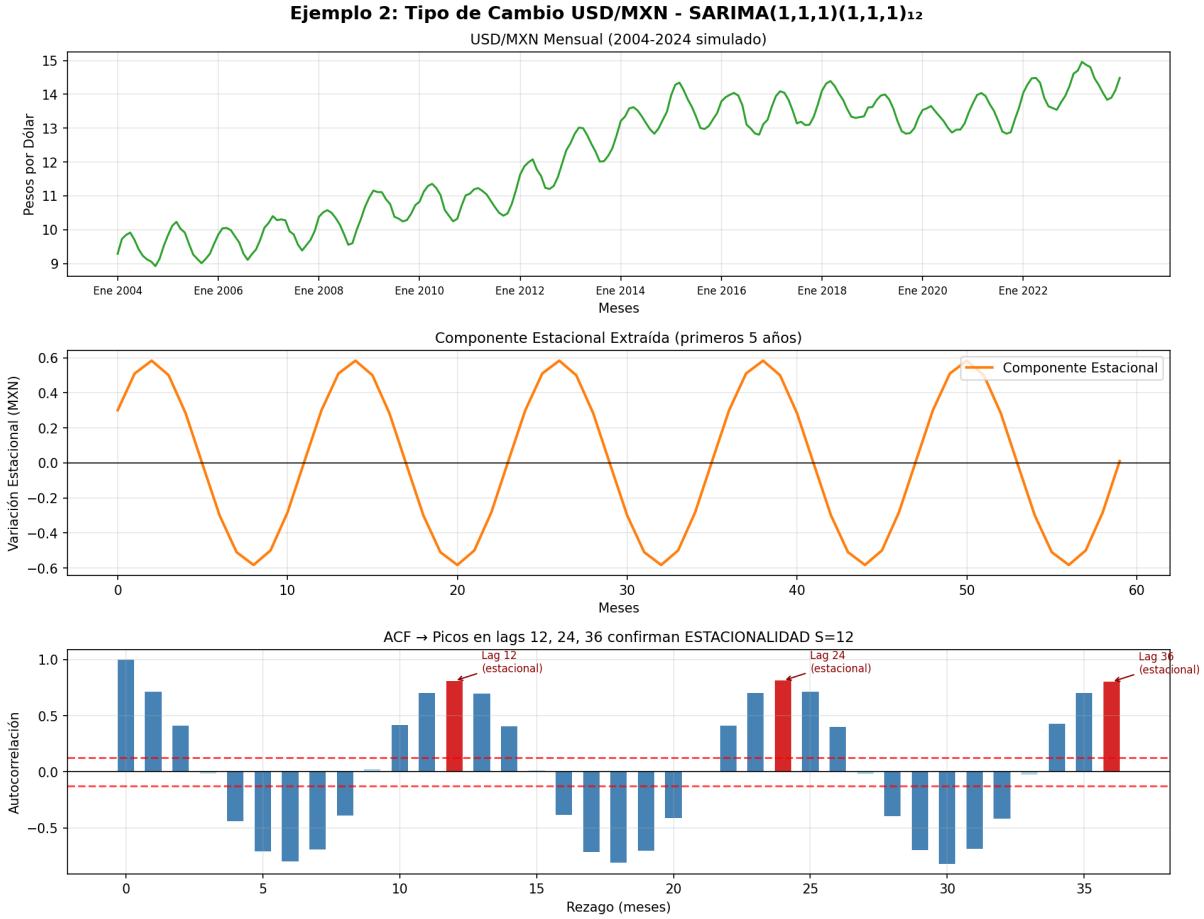


Figure 4: Serie mensual USD/MXN, componente estacional extraída, y ACF con picos estacionales en rezagos 12, 24 y 36.

¿Por qué SARIMA para el tipo de cambio mensual?

- $S = 12$: Datos mensuales con patrones que se repiten anualmente
- $d = 1$, $D = 1$: La serie tiene tendencia (diferenciación regular) y estacionalidad persistente (diferenciación estacional)
- **Picos en ACF a los rezagos 12, 24, 36**: Evidencia clara de estacionalidad anual
- Los componentes $(P, D, Q)_S$ capturan la dinámica entre meses equivalentes de años diferentes (enero de este año vs enero del año pasado)

6.1. Código Python — Ejemplo 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import seasonal_decompose

# ================================================================
# PASO 1: Descargar datos (en produccion):
# import yfinance as yf
# data = yf.download("MXNUSD=X", start="2004-01-01",
#                    end="2024-12-31", interval="1mo")
# usdmxn = (1 / data["Close"]).rename("USD_MXN")
#
# Simulamos datos mensuales con estacionalidad:
# ================================================================
np.random.seed(99)
n_meses = 240  # 20 anos de datos mensuales
t = np.arange(n_meses)

# Componentes de la serie
tendencia = 0.02 * t
estacional = 0.5*np.sin(2*np.pi*t/12) + 0.3*np.cos(2*np.pi*t/12)
ruido = np.cumsum(np.random.normal(0, 0.1, n_meses))

fechas = pd.date_range("2004-01-01", periods=n_meses, freq="ME")
usdmxn = pd.Series(9.0 + tendencia + estacional + ruido,
                   index=fechas, name="USD_MXN")

# PASO 2: Descomposicion estacional
descomp = seasonal_decompose(usdmxn, model="additive", period=12)
descomp.plot()
plt.suptitle("Descomposicion: Tendencia + Estacional + Residual")
plt.tight_layout()
plt.show()

# PASO 3: ACF para identificar estacionalidad
fig, axes = plt.subplots(2, 1, figsize=(12, 6))
plot_acf(usdmxn.diff().dropna(), lags=36, ax=axes[0])
axes[0].set_title("ACF de la 1ra Diferencia")
axes[0].set_xlabel("Rezago (meses)")
# Picos en 12, 24, 36 confirman S=12

plot_pacf(usdmxn.diff().dropna(), lags=36, ax=axes[1])
axes[1].set_title("PACF de la 1ra Diferencia")
axes[1].set_xlabel("Rezago (meses)")
plt.tight_layout()
plt.show()

# PASO 4: Ajustar SARIMA(1,1,1)(1,1,1)_12
# order=(p,d,q) - parte no estacional
# seasonal_order=(P,D,Q,S) - parte estacional
modelo_sarima = SARIMAX(usdmxn,
                        order=(1, 1, 1),
                        seasonal_order=(1, 1, 1, 12),
                        enforce_stationarity=False,
                        enforce_invertibility=False)
resultado_sarima = modelo_sarima.fit(disp=False)
print(resultado_sarima.summary())

# PASO 5: Comparar modelos por AIC y BIC
modelos_candidatos = [
```

```python
    {"order": (1,1,1), "seasonal": (1,1,1,12)},
    {"order": (2,1,1), "seasonal": (1,1,1,12)},
    {"order": (1,1,2), "seasonal": (1,1,0,12)},
]

print("\nComparacion de modelos:")
print(f"{'Modelo':<30} {'AIC':>10} {'BIC':>10}")
print("-" * 52)
for cfg in modelos_candidatos:
    try:
        m = SARIMAX(usdmxn, order=cfg["order"],
                    seasonal_order=cfg["seasonal"],
                    enforce_stationarity=False,
                    enforce_invertibility=False).fit(disp=False)
        nombre = f"SARIMA{cfg['order']}{cfg['seasonal']}"
        print(f"{nombre:<30} {m.aic:>10.2f} {m.bic:>10.2f}")
    except:
        pass

# PASO 6: Pronostico a 24 meses (2 anos)
pronostico = resultado_sarima.get_forecast(steps=24)
pred_media = pronostico.predicted_mean
pred_ic = pronostico.conf_int()

plt.figure(figsize=(14, 5))
plt.plot(usdmxn[-36:], label="USD/MXN historico", color="green")
plt.plot(pred_media, label="Pronostico SARIMA", color="red",
         linestyle="--")
plt.fill_between(pred_ic.index,
                 pred_ic.iloc[:, 0], pred_ic.iloc[:, 1],
                 alpha=0.2, color="red", label="IC 95%")
plt.title("Pronostico USD/MXN - SARIMA(1,1,1)(1,1,1)_12")
plt.ylabel("Pesos por Dolar (MXN)")
plt.legend()
plt.show()
```

Listing 2: Ejemplo 2: SARIMA(1,1,1)(1,1,1)_12 para USD/MXN mensual

7. Ejemplo 3: Precio del Petróleo WTI — ARIMA(2,1,2) con Pronóstico

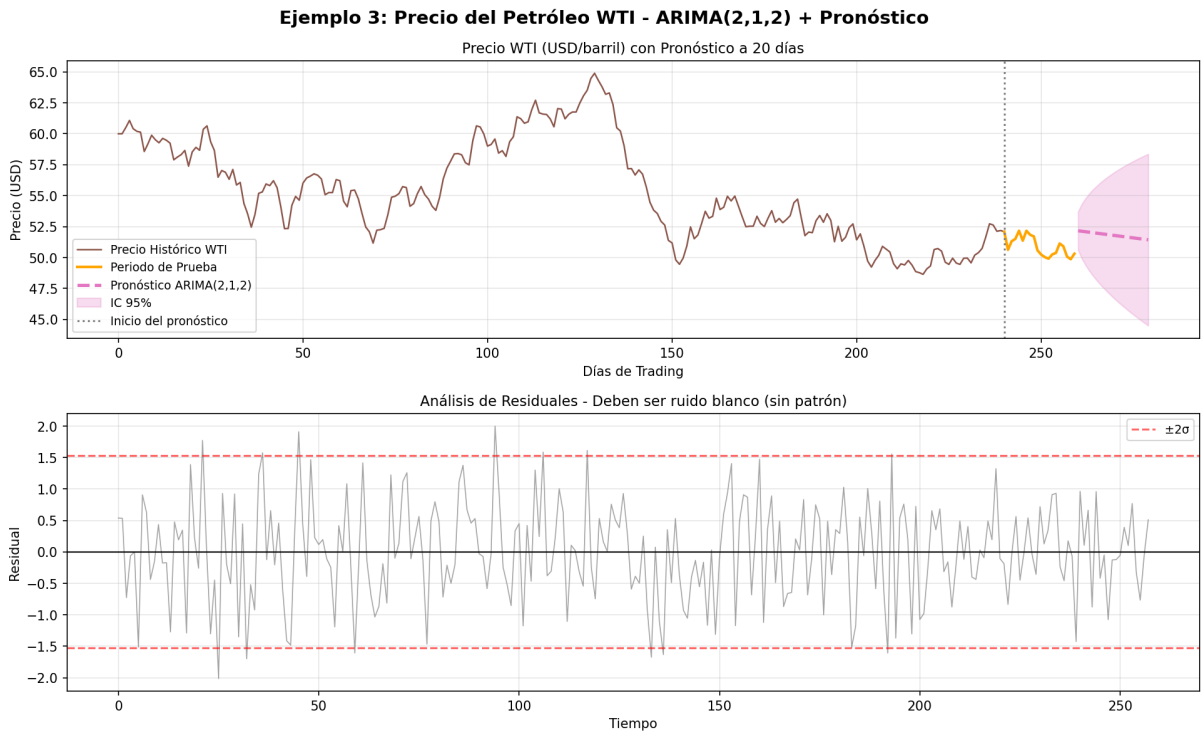


Figure 5: Precio del petróleo WTI, pronóstico a 20 días con intervalos de confianza, y análisis de residuales.

¿Por qué ARIMA(2,1,2) para el petróleo?

- $d = 1$: Como las acciones, el precio del petróleo tiene raíz unitaria (no estacionario)
- $p = 2$: La PACF muestra picos significativos en rezagos 1 y 2, indicando que los dos días anteriores tienen efecto
- $q = 2$: La ACF también muestra autocorrelación en los primeros 2 rezagos
- **Residuales como ruido blanco**: Los residuales correctamente modelados no tienen patrón

7.1. Código Python — Ejemplo 3

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
from itertools import product

# ==============================================================
```

```python
# PASO 1: Descargar datos (en produccion):
# import yfinance as yf
# data = yf.download("CL=F", start="2019-01-01", end="2024-12-31")
# wti = data["Close"]
#
# Simulacion de precio WTI con dinamica ARIMA(2,1,2):
# ==============================================================
np.random.seed(7)
n3 = 260
phi1, phi2 = 0.6, -0.3
theta1, theta2 = -0.4, 0.2

eps3 = np.random.normal(0, 1.5, n3 + 10)
y = np.zeros(n3 + 10)
for t in range(2, n3 + 10):
    y[t] = (phi1*y[t-1] + phi2*y[t-2]
            + eps3[t] + theta1*eps3[t-1] + theta2*eps3[t-2])

oil_returns = y[:n3] * 0.01
wti = pd.Series(60 * np.cumprod(1 + oil_returns), name="WTI")

# PASO 2: Separar entrenamiento y prueba
n_train = 240
train = wti[:n_train]
test = wti[n_train:]

# PASO 3: Busqueda automatica del mejor modelo (grid search)
print("Buscando mejor modelo ARIMA...")
resultados_busqueda = []
for p, q in product(range(4), repeat=2):
    try:
        m = ARIMA(train, order=(p, 1, q)).fit()
        resultados_busqueda.append({
            "p": p, "q": q,
            "AIC": round(m.aic, 2),
            "BIC": round(m.bic, 2)
        })
    except:
        pass

df_resultados = (pd.DataFrame(resultados_busqueda)
                 .sort_values("AIC")
                 .reset_index(drop=True))
print(df_resultados.head(5).to_string(index=False))

# PASO 4: Ajustar el modelo seleccionado ARIMA(2,1,2)
modelo_wti = ARIMA(train, order=(2, 1, 2))
resultado_wti = modelo_wti.fit()
print("\n", resultado_wti.summary())

# PASO 5: Pronostico rolling (mas robusto)
n_fc = len(test)
pronosticos = []
historia = list(train.values)

for i in range(n_fc):
    m_temp = ARIMA(historia, order=(2, 1, 2)).fit()
    pred = m_temp.forecast(steps=1)[0]
```

```python
    pronosticos.append(pred)
    historia.append(test.iloc[i])  # anadir valor real

# PASO 6: Evaluar desempeno
error_mae = np.mean(np.abs(np.array(pronosticos) - test.values))
error_rmse = np.sqrt(np.mean((np.array(pronosticos)-test.values)**2))
print(f"\nMAE:  {error_mae:.4f}")
print(f"RMSE: {error_rmse:.4f}")

# PASO 7: Diagnostico de residuales con Ljung-Box
residuales = resultado_wti.resid
ljung = acorr_ljungbox(residuales, lags=[10, 20], return_df=True)
print("\nPrueba Ljung-Box (H0: no hay autocorrelacion):")
print(ljung)
# Si p-value > 0.05 en todos los lags, los residuales son ruido
    blanco

# PASO 8: Visualizar resultados
fig, axes = plt.subplots(2, 1, figsize=(13, 8))

axes[0].plot(wti.values, color="#8c564b", linewidth=1.2,
             label="WTI Historico")
axes[0].plot(range(n_train, n_train+n_fc), pronosticos,
             color="red", linewidth=2, linestyle="--",
             label="Pronostico Rolling ARIMA(2,1,2)")
axes[0].axvline(n_train, color="gray", linestyle=":", linewidth=1.5)
axes[0].set_title("Pronostico Rolling - Petroleo WTI")
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(residuales.values, color="gray", alpha=0.7)
axes[1].axhline(0, color="black")
axes[1].axhline(1.96*residuales.std(), color="red",
                linestyle="--", label="IC 95%")
axes[1].axhline(-1.96*residuales.std(), color="red", linestyle="--")
axes[1].set_title("Residuales del Modelo - deben ser ruido blanco")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

Listing 3: Ejemplo 3: ARIMA(2,1,2) para petróleo WTI con pronóstico

8. Resumen de Criterios de Diagnóstico

Checklist: ¿Mi modelo SARIMA está bien especificado?

- ✓ **Estacionariedad**: Prueba ADF con p-valor < 0.05 tras diferenciar
- ✓ **ACF/PACF**: El modelo captura la estructura de autocorrelación
- ✓ **Residuales**: No tienen autocorrelación (Ljung-Box $p > 0.05$)
- ✓ **Normalidad**: Los residuales siguen distribución normal (Jarque-Bera)
- ✓ **AIC/BIC**: El modelo elegido tiene el menor valor entre candidatos
- ✓ **Parsimonia**: Si dos modelos tienen AIC similar, elegir el más simple

Prueba	Hipótesis nula	Queremos
Dickey-Fuller (ADF)	La serie tiene raíz unitaria	$p < 0.05$ para diferenciar
Ljung-Box	Los residuales son ruido blanco	$p > 0.05$ en todos los lags
Jarque-Bera	Los residuales son normales	$p > 0.05$

Para usar yfinance en producción

En un ambiente real con internet, instala:

```
pip install yfinance statsmodels
```

Y reemplaza la sección de simulación por:

```
import yfinance as yf
data = yf.download("AAPL", start="2020-01-01", end="2024-12-31")
precio = data["Close"]
```

Los mismos pasos del análisis aplican exactamente igual.

— Fin del Documento —

Guía de Modelos SARIMA para Series de Tiempo Financieras